ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Engineering (CSE)



System Design and Architecture Report

SWE 4404: Software Project Lab II

Project Title

WanderVille

Name of the Supervisor

Md. Tariquzzaman

Team Members

Safia Aman Anika	230042120
Musfeka Ferdousi	230042126
Khadiza Sultana	230042135

Table of Contents

1. Introduction **2**

2. Purpose of the Game **2**

4. SRS Document: **3**

5. Class Diagram **3**

6. Three-layer architecture diagram **6**

Domain 1 - Authentication & Core Session 6

Domain 2 - World, Farming & Environment 6

Domain 3 - Inventory, Economy & Shop 7

Domain 4 - Fishing, Pets & Social 8

Domain 5 - Achievements, Daily Tasks & Notifications 8

7. Conclusion **9**

1. Introduction

WanderVille is a cozy 2D isometric multiplayer farming and social simulation game. The project blends familiar life-simulation mechanics - farming, crafting, decorating, and socializing - with a setting that is personally meaningful to its target audience: university students who want a relaxing, community-driven escape that still feels like "home."

2. Purpose of the Game

The primary purpose of WanderVille is to provide a low-stress, socially engaging virtual space where students can unwind, connect with peers, and experience a nostalgic, stylized version of their own campus. Unlike competitive or high-tension games, WanderVille is designed around cooperative and casual interaction - players tend farms, build relationships with in-game and real-world friends, and gradually shape a shared virtual village together. The game also serves as a creative outlet for self-expression through customization (avatars, homes, farm plots) and a light social simulation layer that mirrors everyday campus life.

3. Feature Summary

1. **Account & Login:** Players can create an account, log in with saved credentials, remain logged in across sessions on the same device, and log out securely.
2. **Movement**: Players move their character in 8 directions (including diagonals), collide with solid objects (buildings, trees, water), and explore the world with a camera that follows them smoothly.
3. **Farming**: Players till soil, plant seeds, water crops, and watch them grow through visible stages at a rate affected by the current season; unwatered crops pause growth instead of dying, and players harvest mature crops for use or sale.
4. **Fishing**: Players fish at marked lake spots using a timing-based mini-game, occasionally collecting rare materials or rare fish.
5. **Housing**: Each player owns a personal house that can be upgraded through fixed tiers using Aura currency, improving its appearance.
6. **Inventory & Currency**: Items are organized into fixed inventory categories (Seeds, Crops, Fish, Rare Materials, Tools, Houses); players earn and spend Aura as the game's sole currency to buy, sell, and manage items.

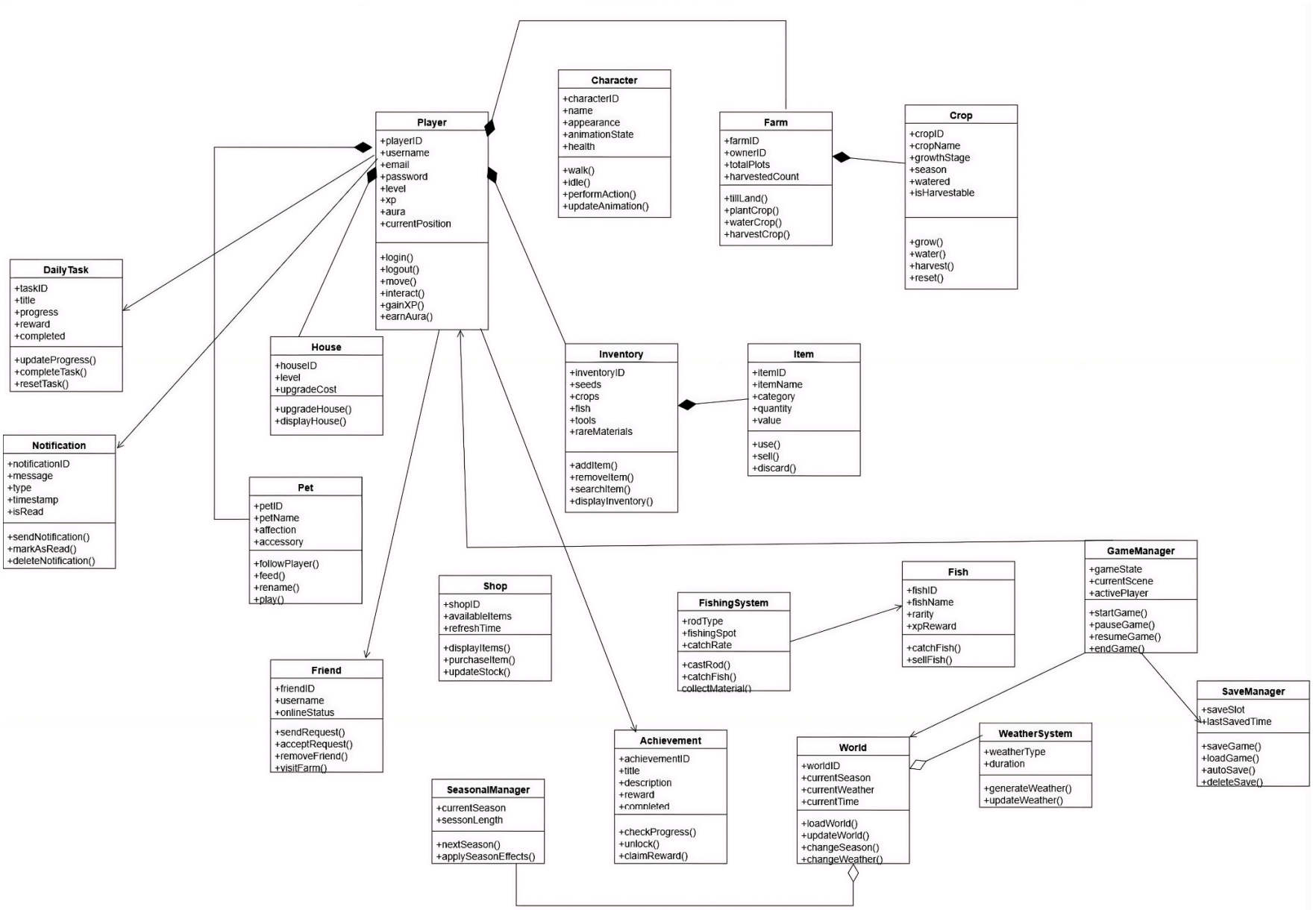
7. **Pets**: Each player is assigned a starter pet that follows them around the world; players can feed, name, and bond with their pet through simple interactions.
8. **World Environment**: The game cycles through four seasons and a day/night cycle, with dynamic weather that affects visuals and gameplay pacing without ever blocking play.
9. **Achievements & Leveling:** The system tracks player progress through achievements and levels, rewarding players with Aura/XP and unlocking new content as they advance.
10. **Friends & Social**: Players can add, accept, or remove friends, visit a friend's farm or map, and view basic profile information to stay connected with others.
11. **Multiplayer Presence**: Other players' movement is displayed in real time within shared world spaces, making the world feel alive and populated.
12. **Gifting & Trading**: Players can send gifts to friends and propose item-for-item trades, with both inventories updating immediately once a trade is completed.
13. **Chat & Communication**: Players can chat with nearby players, message friends directly, and use emotes to socialize within the game world.

4. SRS Document:

https://docs.google.com/spreadsheets/d/1Q6M5NY0QgDIuwNP6kxEd7jx_kQc_CkXyDkbsuCCh_rk/edit?gid=0#gid=0

5. Class Diagram

The Class Diagram represents the static structure of the game. It illustrates the classes, their attributes, methods, and relationships. The Player class interacts with the Inventory, Farm, Pet, and Achievement classes, while the GameManager coordinates the overall gameplay.


Character
+characterID
+name
+appearance
+animationState
+health
+walk()
+idle()
+performAction()
+updateAnimation()
Player
+playerID
+username
+email
+password
+level
+xp
+aura
+currentPosition
+login()
+logout()
+move()
+interact()
+gainXP()
+earnAura()
Farm
+farmID
+ownerID
+totalPlots
+harvestedCount
+tillLand()
+plantCrop()
+waterCrop()
+harvestCrop()
Crop
+cropID
+cropName
+growthStage
+season
+watered
+isHarvestable
+grow()
+water()
+harvest()
+reset()
DailyTask
+taskID
+title
+progress
+reward
+completed
+updateProgress()
+completeTask()
+resetTask()
House
+houseID
+level
+upgradeCost
+upgradeHouse()
+displayHouse()
Inventory
+inventoryID
+seeds
+crops
+fish
+tools
+rareMaterials
+addItem()
+removeItem()
+searchItem()
+displayInventory()
Item
+itemID
+itemName
+category
+quantity
+value
+use()
+sell()
+discard()
Notification
+notificationID
+message
+type
+timestamp
+isRead
+sendNotification()
+markAsRead()
+deleteNotification()
Pet
+petID
+petName
+affection
+accessory
+followPlayer()
+feed()
+rename()
+play()
Shop
+shopID
+availableItems
+refreshTime
+displayItems()
+purchaseItem()
+updateStock()
FishingSystem
+rodType
+fishingSpot
+catchRate
+castRod()
+catchFish()
collectMaterial()
Fish
+fishID
+fishName
+rarity
+xpReward
+catchFish()
+sellFish()
GameManager
+gameState
+currentScene
+activePlayer
+startGame()
+pauseGame()
+resumeGame()
+endGame()
Friend
+friendID
+username
+onlineStatus
+sendRequest()
+acceptRequest()
+removeFriend()
+visitFarm()
Achievement
+achievementID
+title
+description
+reward
+completed
+checkProgress()
+unlock()
+claimReward()
World
+worldID
+currentSeason
+currentWeather
+currentTime
+loadWorld()
+updateWorld()
+changeSeason()
+changeWeather()
WeatherSystem
+weatherType
+duration
+generateWeather()
+updateWeather()
SaveManager
+saveSlot
+lastSavedTime
+saveGame()
+loadGame()
+autoSave()
+deleteSave()
SeasonalManager
+currentSeason
+sessonLength
+nextSeason()
+applySeasonEffects()

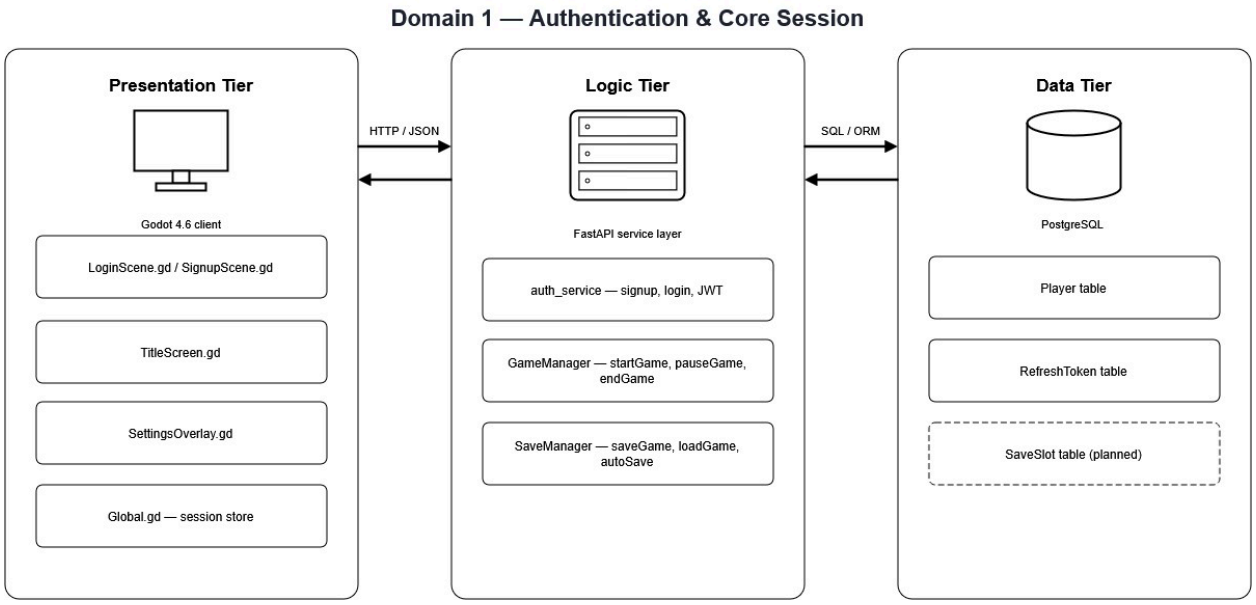
6. Three-layer architecture diagram

Wanderville uses a standard three-tier architecture: a Presentation Tier (Godot 4.6 client), a Logic Tier (FastAPI service layer), and a Data Tier (PostgreSQL via SQLAlchemy). The client never touches the database directly - every request goes through a service module that validates it and applies game rules before anything is read or written. This keeps the client swappable and the rules independently testable.

Since the project isn't finished, the architecture is split into five domains matching groups of functional requirements from the SRS. Solid boxes are already implemented; dashed/greyed boxes are designed but not yet built, so the diagrams also show current progress honestly.

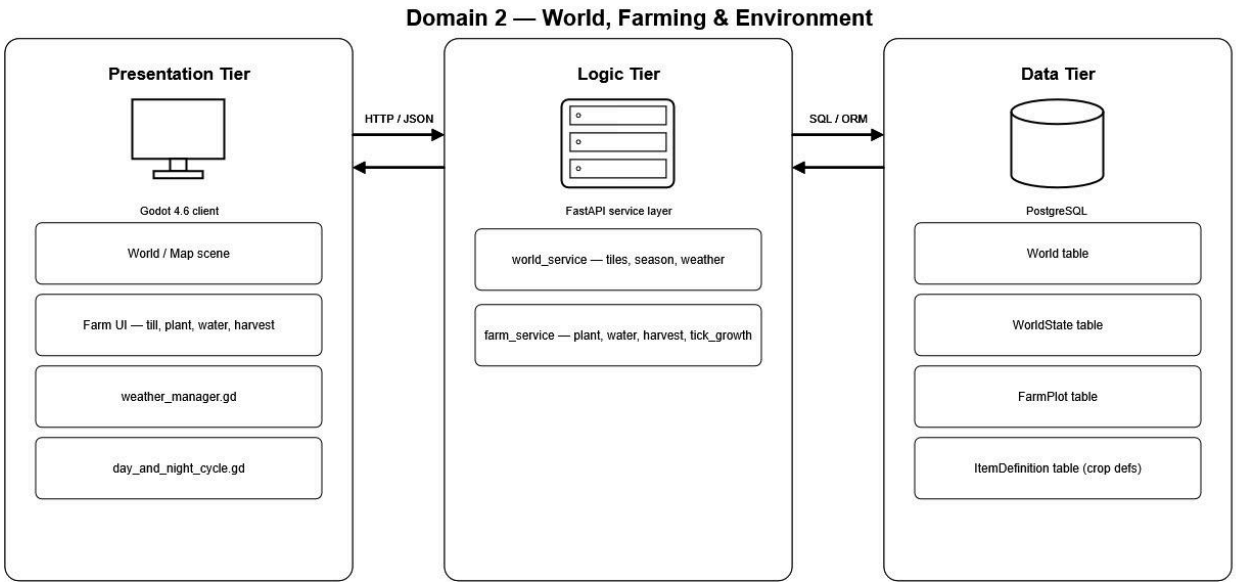
Domain 1 - Authentication & Core Session

Handles account creation, login, session persistence, and save/load, backed by the Player and RefreshToken tables.



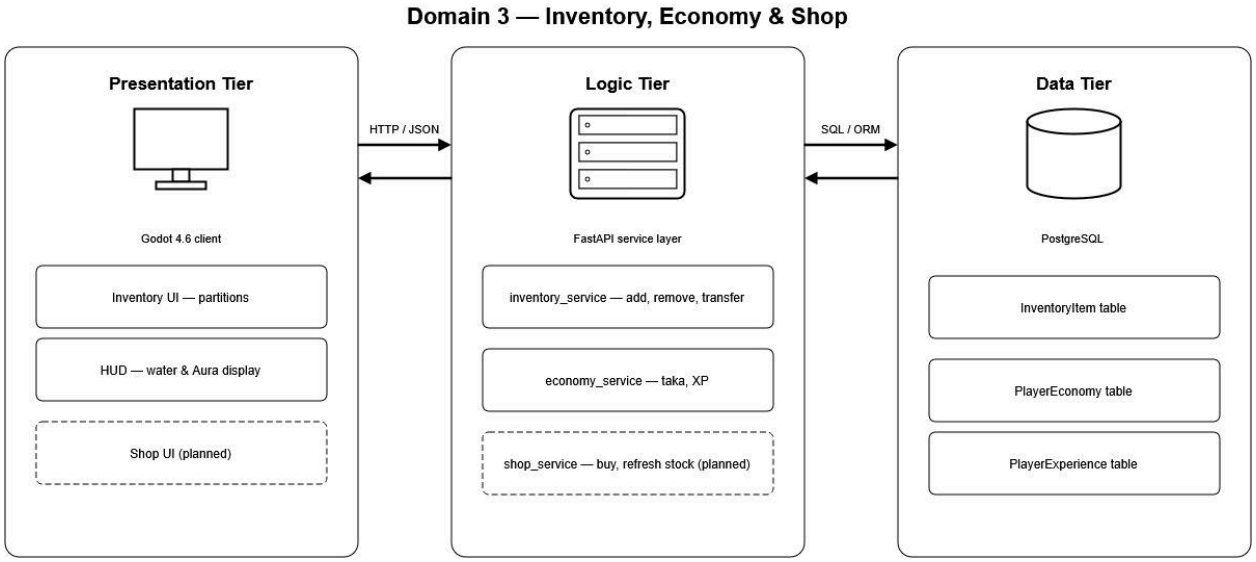
Domain 2 - World, Farming & Environment

Manages the shared map, tile-based farming (till/plant/water/harvest), and the season/weather cycle that affects crop growth.



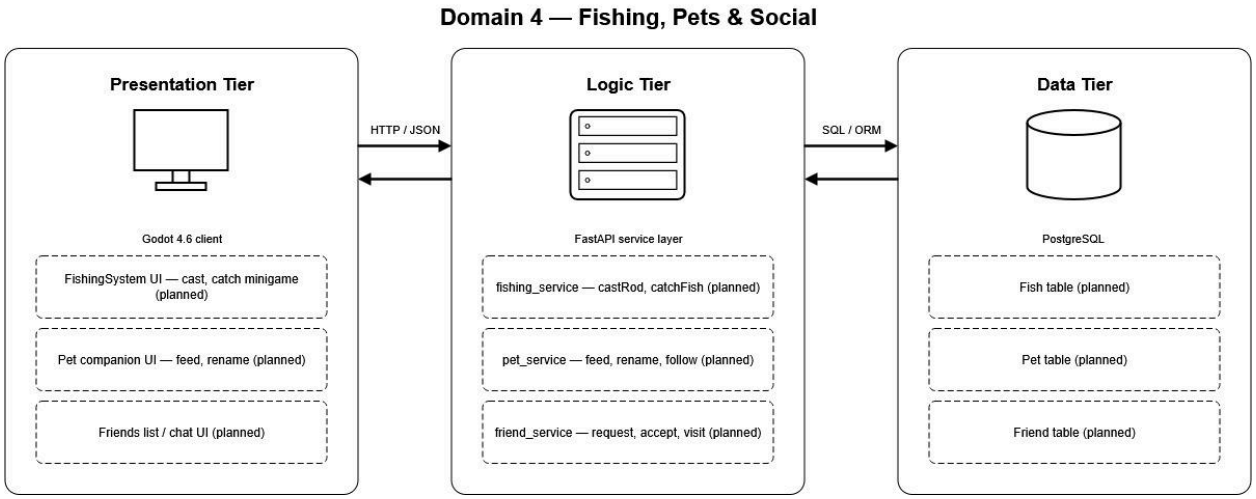
Domain 3 - Inventory, Economy & Shop

Covers item storage across partitions, the Aura currency and XP system, and shop purchases (shop still planned).



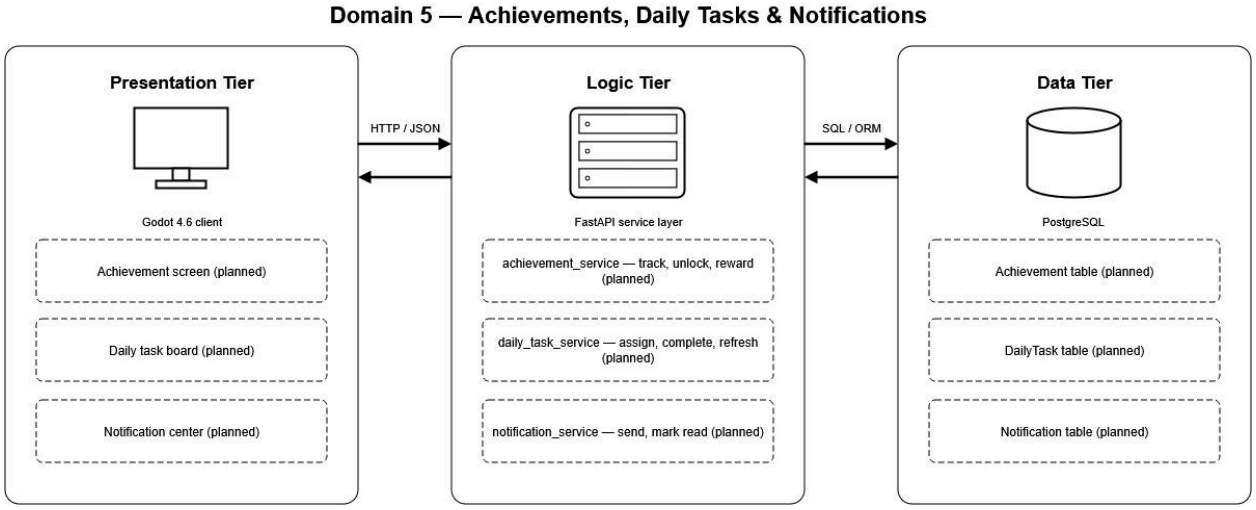
Domain 4 - Fishing, Pets & Social

Covers the fishing mini-game, pet companion, and friends/chat features — fully designed in the SRS but not yet implemented



Domain 5 - Achievements, Daily Tasks & Notifications

Covers player progression loops and in-game alerts (achievements, daily tasks, notifications) — also planned but not yet built.



7. Conclusion

WanderVille aims to give university students a cozy, multiplayer farming and social space built around a campus setting they can recognize. The core gameplay loop — authentication, movement, and farming — is implemented on a working FastAPI + PostgreSQL backend and Godot 4.6 client. The remaining domains — fishing, pets, social features, and progression/notifications — are fully specified in the SRS but not yet built. The three-tier architecture was chosen specifically so these domains can be added incrementally without disrupting what's already working. Going forward, the team will prioritize pets and fishing next, since they extend the existing farming loop directly, followed by social and notification features once multiplayer presence is stable.